📖 GRAPH PATTERN GUIDE

PAGE 1 — GRAPH FOUNDATIONS

"Think like a FAANG interviewer before writing a single line of code."

1 PLAIN ENGLISH

Graphs model relationships.

Most interview problems hide graphs.

Before coding, recognize the hidden graph, understand its type, choose the right representation and the right pattern.

3 BRAIN SWITCH

Interview gives...

- 🌆 Cities & Roads
- 👥 Friends / Social Network
- ✈️ Flights / Airlines
- 📚 Courses / Prerequisites
- 💻 Computer Network
- 🏁 Maze / Game Map
- 🏗️ Dependencies / Tasks
- 📁 Files / Folders Structure

➡️ **THINK GRAPH**

Whenever you see relationships, think GRAPH.

2 VISUAL EXAMPLES

TREE (Special Case of Graph)

No cycles, Connected, Has a root.

GENERAL GRAPH

May have cycles,
May be disconnected.

GRID (2D)

Cells are nodes,
Neighbors are edges.

4 GRAPH TYPES

UNDIRECTED

Edges have no direction.
Ex: Friends

DIRECTED

Edges have direction.
Ex: Flights

WEIGHTED

Edges have weights/costs.
Ex: Roads

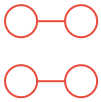
UNWEIGHTED

All edges have equal weight.
Ex: Social Network

CONNECTED

There is a path between every pair.
Ex: Network

DISCONNECTED



Not all nodes are reachable.
Ex: Components

CYCLIC

Contains at least one cycle.
Ex: Dependencies

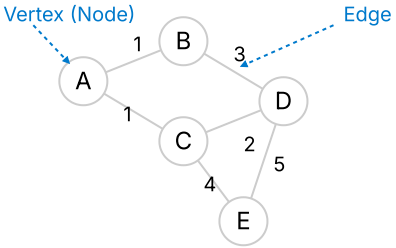
ACYCLIC

No cycles.
Ex: Tree

DAG (DIRECTED ACYCLIC GRAPH)

Directed and no cycles.
Ex: Course Schedule

5 GRAPH TERMINOLOGY



Degree (A) = 2 : Edges connected to A.
Path: A → C → E (Sequence of vertices)
Cycle: B → C → D → B (Start and end at same node)
Connected Component: Set of nodes that are reachable.

6 TREE vs GRAPH

Aspect	TREE	GENERAL GRAPH
Root	Has a root	No root (usually)
Parent	Each node (except root) has exactly one parent	A node can have multiple parents
Neighbor	Limited by parent-child relationship	Any connected node is a neighbor
Cycles	No cycles	May have cycles
Connected	Always connected	May be connected or disconnected
Edges	For n nodes, edges = n - 1	Edges can be any number
Traversal	DFS / BFS works	DFS / BFS works
Need visited[]	Usually Not Needed	Usually Needed
Interview Examples	Binary Tree, N-ary Tree	Social Network, Roads, Flights, Dependencies

A Tree is always a Graph.

8 AHA MOMENTS

💡
A Tree is a Graph.

💡
A Grid is a Graph.

💡
Representation changes. Algorithm usually doesn't.

💡
Most interview problems hide the graph.

💡
Outer loop = Multiple Components.

7 GRAPH REPRESENTATIONS (OVERVIEW)

Representation	Visual	Interview Usage	Memory	Notes
Edge List	List of edges [u, v, w]	Kruskal, Some Problems	O(E)	Simple to read
Adjacency List	[1: [2, 3], 2: [1, 3], ...]	**Most Common in Interviews**	O(V + E)	★★★★★ **95% Interview Problems**
Adjacency Matrix	2D Matrix (n x n)	Dense Graphs, Quick Edge Check	$O(V^2)$	Good for dense graphs
Grid	2D Characters / Integers	Islands, Paths, Flood Fill	O(m x n)	Grid treated as Graph
Node Graph	Nodes with Neighbors	Clone Graph, Complex Structures	O(V + E)	Used in advanced problems

Adjacency List is the most commonly used representation in interviews.

9 PATTERN RECOGNITION

Interview Says...	Think	Expected Graph Type
🌆 Cities & Roads	Graph	Undirected, Weighted or Unweighted
✈️ Flights / Airlines	Graph	Directed, Weighted
📚 Courses / Prerequisites	Graph	Directed, DAG
🏁 Grid / Matrix	Graph	Grid Graph (4 / 8 direction)
💻 Computer Network	Graph	Undirected or Directed
👥 Friends / Social Network	Graph	Undirected, Unweighted
🗺️ Maze / Game Map	Graph	Grid Graph
🏗️ Dependencies / Build Order	Graph	Directed, DAG
📁 File / Folder Structure	Graph	Tree (Special Graph)

10 COMMON INTERVIEW MISTAKES

- ❌ Forgetting graphs may be disconnected.
- ❌ Confusing Tree with Graph.
- ❌ Ignoring direction (Directed vs Undirected).
- ❌ Ignoring weights.
- ❌ Choosing wrong representation.
- ❌ Started coding before deciding representation.
- ❌ Not defining neighbors clearly.
- ❌ Forgetting to validate with small examples.

11 INTERVIEW THINKING (BEFORE CODING)

1. What is the input?
Understand the given data.

2. Is it Directed?
Do edges have direction?

3. Is it Weighted?
Do edges have weights/costs?

4. Is it Connected?
Can all nodes reach other nodes?

5. Which Representation should I build?
Adj List, Matrix, Grid, etc.

6. Which Pattern family will probably solve it?
DFS, BFS, Topological, Union Find, Dijkstra, etc.

❓ Problem (Understand) ➔ 📥 Input (What given) ➔ 🧱 Representation (How to model) ➔ 🛠️ Helper Structure (visited, parent, etc.) ➔ ⚙️ Pattern (DFS / BFS / etc.) ➔ 🔧 Tiny Logic Change (Per Problem) ➔ ▶️ Dry Run (Example) ➔ ⏱️ Optimize (Time/Space)

📖 GRAPH PATTERN GUIDE

PAGE 2 — GRAPH INPUT TYPES & CONSTRUCTION GUIDE

"Every graph interview begins by converting the input into the right representation."

1 PLAIN ENGLISH

🎯 Interviewers rarely give you a graph directly.
Instead they give edges, grids, matrices or node objects.
💡 Your first job is to identify the input and convert it into the correct graph representation.

3 BRAIN SWITCH

INTERVIEW INPUT

🎀 Edge List	→ Build AdjList
⚖️ Weighted Edge List	→ Build Weighted AdjList
🏁 Grid	→ Treat Cell as Node
🧮 Adjacency Matrix	→ Convert or Traverse Directly
🔗 Node Graph	→ Use Existing Neighbors

REPRESENTATION FIRST. ALGORITHM SECOND.

2 INPUT TYPES (What Interviewers Give You)

1

EDGE LIST

Example Input

```
[[0,1], [1,2], [2,3]]
```

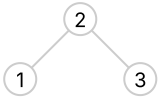
Typical Problems
- Valid Path
- Connected Components
- Graph Valid Tree

★★★★★

WEIGHTED EDGE LIST

Example Input

```
[[2,1,1], [2,3,1],
[3,4,1]]
```



Typical Problems
- Network Delay
- Cheapest Flights
- Minimum Effort Path

★★★★★

GRID

Example Input

```
1 1 0 1 0
1 1 1 0 0
```

1	1	0
1	1	1

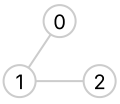
Typical Problems
- Number of Islands
- Flood Fill
- Rotting Oranges
- Pacific Atlantic

★★★★★

ADJACENCY MATRIX

Example Input

```
0 1 0
1 0 1
0 1 0
```

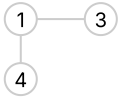


Typical Problems
- Number of Provinces
- Dense Graph Problems

★★☆☆☆

NODE GRAPH (OBJECTS)

Example Input
Node Neighbors[]



Typical Problems
- Clone Graph

★★☆☆☆

4 GRAPH CONSTRUCTION GUIDE

Input (Interview Gives)	How to Build	Output (Your Representation)	Notes
Edge List `[[u, v], ...]`	**Build Adjacency List** (Undirected / Directed)	`List<List<Integer>>`	Most common representation. Optimal for sparse graphs.
Weighted Edge List `[[u, v, w], ...]`	**Build Weighted Adjacency List**	`List<List<int[]>>` Each int[] = {neighbor, weight}	Use `int[]` or `Pair<node, weight>`
Grid `matrix[m][n]`	Use directions (`dirs[]`) Treat valid cell as node	**Implicit Graph** (no explicit structure needed)	Neighbor checks on the fly using dirs[]
Adjacency Matrix `matrix[n][n]`	Use matrix directly or convert to AdjList	`int[][] matrix` (or convert to `List<List<Integer>>`)	Good for dense graphs. O(1) edge check.
Node Graph Node object with neighbors[]	Use existing neighbors directly	**Node (given)** (no conversion needed)	Already a graph. Just use it.

Adjacency List is the preferred choice for 95% of interview problems.

5 VISUAL CONVERSION EXAMPLES

1 Edge List → Adjacency List

Edge List	Adjacency List
[[0,1], [0,2], [1,2], [2,3]]	0 → [1,2] 1 → [0,2] 2 → [0,1,3] 3 → [2]

2 Weighted Edge → Weighted AdjList

Weighted Edges	Weighted AdjList
[[0,1,4], [0,2,5], [1,2,1]]	0 → [(1,4), (2,5)] 1 → [(0,4), (2,1)] 2 → [(0,5), (1,1)]

3 Grid → Implicit Graph

1	1
1	1

→

4 Matrix → Graph

0	1
1	0

→

5 Node Objects → Graph

Nodes (given) → Neighbor Links (Already Built)

6 BUILD ➔ PRINT ➔ VALIDATE (Never Skip This!)

Understand Input ➔ **Build Graph** ➔ **Print Graph** ➔ **4 Validate Structure** ➔ **Start Algorithm**

How to Validate Your Graph

Graph Type	How to Validate
Adjacency List	Print each node and its neighbors
Weighted Graph	Print each node with (neighbor, weight)
Grid	Print grid dimensions and values
Matrix	Print the entire matrix
Node Graph	Print each node and neighbor count

⚠️ **IMPORTANT:** Never start DFS/BFS (or any algorithm) until your graph structure is verified.

7 COMMON JAVA BUILD TEMPLATES (Construction Only)

● Build Adjacency List

```
// 1. Build Adjacency List // List<List<Integer>> int n = V; // number of nodes List<List<Integer>> graph = new ArrayList<>(); for (int i = 0; i < n; i++) graph.add(new ArrayList<>()); // For undirected edge (u, v) graph.get(u).add(v); graph.get(v).add(u);
```

● Build Weighted AdjList

```
// 2. Build Weighted Adjacency List // List<List<int[]>> int n = V; List<List<int[]>> graph = new ArrayList<>(); for (int i = 0; i < n; i++) graph.add(new ArrayList<>()); // Edge (u, v, w) graph.get(u).add(new int[]{v, w}); graph.get(v).add(new int[]{u, w});
```

● Directions Array

```
// 3. Directions Array (For Grid) // int[][] dirs int[][] dirs = { { 1, 0}, // down {-1, 0}, // up { 0, 1}, // right { 0, -1} // left };
```

8 AHA MOMENTS

💡 The algorithm usually doesn't change.

💡 Only the graph representation changes.

💡 A Grid is an implicit graph.

💡 Weighted graphs store (neighbor, weight).

💡 Node graphs are already built.

💡 Always build before solving.

9 COMMON INTERVIEW MISTAKES

- ❌ Started coding before building the graph.
- ❌ Forgot reverse edge in undirected graph.
- ❌ Used wrong representation.
- ❌ Stored weight incorrectly.
- ❌ Forgot 1-index vs 0-index.
- ❌ Didn't validate the graph.
- ❌ Mixed node and weight positions.

10 INTERVIEW THINKING FLOW

❓ **Problem** ➔ 📥 **What input did interviewer give?** ➔ 🔗 Edge List / ⚖️ Weighted Edge / 🏁 Grid / 🧮 Matrix / 🔴 Node Graph ➔ 🎯 **Ready for Algorithm**

📝 Choose Representation ➔ 🏗️ Build Graph ➔ 🖨️ Print Graph ➔ ✅ Validate Structure

11 SUMMARY

- ✅ **Input** determines Representation.
- ✅ **Representation** determines Data Structure.
- ✅ **Data Structure** enables Algorithm.
- ✅ Never skip graph construction.

❓ Problem (Understand) ➔ 📥 Input (What given) ➔ 🧱 Representation (How to model) ➔ 🛠️ Helper Structure (visited, parent, etc.) ➔ ⚙️ Pattern (DFS / BFS / etc.) ➔ 🔧 Tiny Logic Change (Per Problem) ➔ ▶️ Dry Run (Example) ➔ ⏱️ Optimize (Time/Space)

📖 GRAPH PATTERN GUIDE

PAGE 3 — GRAPH HELPER STRUCTURES & INTERVIEW TOOLKIT

"Before choosing an algorithm, choose the helper structures that power it."

1 PLAIN ENGLISH

🎯 Almost every graph problem uses helper structures.
💡 Choosing the right helper structure is often more important than choosing the algorithm.
🏗️ **Build first. Choose helper structures. Then solve.**

3 WHEN DO I NEED WHAT?

👁️ Need to avoid revisiting nodes?➔ `visited[]`

🗺️ Need shortest distance?➔ `dist[]`

⏳ Need smallest distance first?➔ `PriorityQueue`

🌊 Need BFS (level by level)?➔ `Queue`

🤝 Need to merge / union groups?➔ `parent[]` , `rank[]`

📊 Need dependency count?➔ `indegree[]`

🧭 Need grid movement?➔ `dirs[][]`

🔗 Need clone / mapping / lookup?➔ `HashMap`

2 THE GRAPH TOOLBOX (Helper Structures You'll Use in Almost Every Problem)

Structure	Purpose (One-line)	Used In (Typical Problems)	Java Type	Initialization (Example)	Validation / What to Check
★ **visited[]**	Track visited nodes	DFS, BFS, Cycle Detect, Components	`boolean[]`	`boolean[] visited = new boolean[n];`	Print visited nodes (true indices)
★ **dist[]**	Store shortest distance from source	Shortest Path, Dijkstra, Multi-Source BFS	`int[]`	`int[] dist = new int[n]; Arrays.fill(dist, INF);`	Print distances (Arrays.toString)
parent[]	Store parent / predecessor of each node	Path Reconstruction, Union Find, Trees	`int[]`	`int[] parent = new int[n]; for(int i=0;i<n;i++) parent[i] = i;`	Print parent array
rank[]	Rank / size for Union Find (Optimized Merge)	Union Find (Disjoint Set)	`int[]`	`int[] rank = new int[n]; // default 0`	Print rank array
indegree[]	Count incoming edges	Topological Sort (Kahn's Algo)	`int[]`	`int[] indeg = new int[n];`	Print indegree array
dirs[][]	Directions for grid movement	Grid DFS/BFS, Islands, Flood Fill	`int[][]`	`int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};`	Print dirs to verify movement
★ **Queue**	Process nodes in FIFO order	BFS, Level Order, Kahn's Algo	`Queue<Integer>`	`Queue<Integer> q = new LinkedList<>();`	Print queue content
★ **PriorityQueue**	Process best candidate first	Dijkstra, Prim's, Minimum Effort	`PriorityQueue<int[]>`	`PriorityQueue<int[]> pq = new PriorityQueue<>();`	Print pq content
HashMap	Mapping / Lookup (Visited, Clone, etc.)	Clone Graph, Store Mappings, Count	`HashMap<K,V>`	`HashMap<K,V> map = new HashMap<>();`	Print map entries

★ Most Frequently Used in Interviews | **Adjacency List + visited[] + Queue + PriorityQueue = Your Core Toolkit**

4 INITIALIZATION CHEAT SHEET

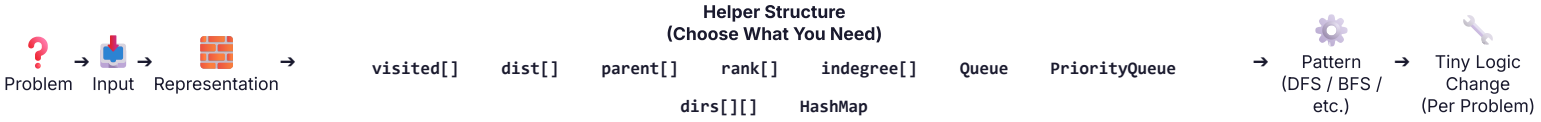
Structure	Java Initialization
visited[]	`boolean[] visited = new boolean[n];`
dist[]	`int[] dist = new int[n]; Arrays.fill(dist, INF);`
parent[]	`int[] parent = new int[n]; for(int i=0; i<n; i++) parent[i] = i;`
rank[]	`int[] rank = new int[n]; // default 0`
indegree[]	`int[] indegree = new int[n];`
Queue	`Queue<Integer> q = new LinkedList<>();`
PriorityQueue	`PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> a[0]-b[0]);`
HashMap	`HashMap<K, V> map = new HashMap<>();`

5 PRINT & VALIDATE (Always Do This!)

Structure	How to Print	Expected Output
visited[]	`Arrays.toString(visited)`	[true, false, true, ...]
dist[]	`Arrays.toString(dist)`	[0, 2, 5, INF, ...]
parent[]	`Arrays.toString(parent)`	[0, 0, 1, 2, ...]
rank[]	`Arrays.toString(rank)`	[0, 1, 0, 1, ...]
indegree[]	`Arrays.toString(indegree)`	[0, 1, 2, 0, ...]
Queue	`System.out.println(q)`	[1, 2, 3, ...]
HashMap	`System.out.println(map)`	{key1=val1, ...}

⚠️ **Never debug DFS/BFS until helper structures look correct.**

6 VISUAL MEMORY MAP (How Everything Connects)



7 AHA MOMENTS

💡 Algorithms usually share helper structures.

💡 Most DFS problems differ only by return type.

💡 Most BFS problems differ only by queue logic.

💡 Grid problems always reuse dirs[].

💡 Weighted graphs almost always need dist[].

💡 PriorityQueue usually means "best candidate first".

8 COMMON INTERVIEW MISTAKES

- ❌ Forgot visited[].
- ❌ Forgot Arrays.fill(dist, INF).
- ❌ Mixed node and distance in structures.
- ❌ Forgot parent[i] = i.
- ❌ Forgot reverse edge in undirected graph.
- ❌ Used Queue instead of PriorityQueue.
- ❌ Forgot dirs[].
- ❌ Didn't print structures while debugging.

9 INTERVIEW THINKING FLOW (Before Coding)

Problem

⬇

Input

⬇

Representation

⬇

Which helper structures do I need?

⬇

Pattern (DFS / BFS / Dijkstra / UF / etc.)

10 SUMMARY

REMEMBER

- ✅ Graph interviews are not won by memorizing algorithms.
- ✅ They are won by choosing the correct helper structures before coding.
- ✅ **Build ➔ Helper Structures ➔ Pattern ➔ Tiny Change ➔ Solve.**
- ✅ **Helper structures are your superpower.**

🧰 **Your Toolkit. Use it Every Time.**

★★★★★

❓ Problem (Understand) ➔ 📥 Input (What given) ➔ 🧱 Representation (How to model) ➔ 🛠️ Helper Structure (visited, parent, etc.) ➔ ⚙️ Pattern (DFS / BFS / etc.) ➔ 🔧 Tiny Logic Change (Per Problem) ➔ ▶️ Dry Run (Example) ➔ ⏱️ Optimize (Time/Space)

📖 GRAPH PATTERN GUIDE

PAGE 4 — MASTER DFS TEMPLATE GUIDE

"One DFS algorithm. Five reusable templates."

1 MASTER IDEA

★★★★★

💡

Almost every Graph DFS interview problem uses one of five reusable templates.

The DFS traversal almost never changes.

Only the caller, input, return type and tiny logic change.

2 GRAPH DFS TEMPLATE

★★★★★

Input	Adjacency List
Caller	`dfs(start)`
Visited	`boolean[]`
Return	`void` / `boolean` / `int`
Used In	• Valid Path • Connected Components • Graph Valid Tree

Java Template

```
private void dfs(int node, List<List<Integer>> graph, boolean[] visited) { visited[node] = true; // mark visited for (int neighbor : graph.get(node)) { // explore neighbors if (!visited[neighbor]) { dfs(neighbor, graph, visited); } } }
```

★★★★★

Input	Grid
Caller	`dfs(r, c)`
Visited	`boolean[][]`
Movement	`dirs[][]`
Used In	• Number of Islands • Flood Fill • Max Area of Island • Surrounded Regions

Java Template

```
private void dfs(int r, int c, char[][] grid, boolean[][] visited) { int m = grid.length, n = grid[0].length; // 1. Boundary check if (r < 0 || r >= m || c < 0 || c >= n) return; // 2. Visited or blocked check (assume '0' or '#' is blocked) if (visited[r][c] || grid[r][c] == '0' || grid[r][c] == '#') return; // 3. Mark visited visited[r][c] = true; // 4. Explore 4 directions int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}}; for (int[] d : dirs) { int nr = r + d[0]; int nc = c + d[1]; dfs(nr, nc, grid, visited); } }
```

★★★★☆

Input	Node (with neighbors)
Caller	`dfs(node)`
Visited	`HashMap<Node, Node>`
Return	Node (cloned node)
Used In	• Clone Graph

Java Template

```
private Node dfs(Node node, HashMap<Node, Node> visited) { if (node == null) return null; if (visited.containsKey(node)) { return visited.get(node); // already cloned } // 1. Create clone for current node Node clone = new Node(node.val); visited.put(node, clone); // 2. Clone all neighbors for (Node nei : node.neighbors) { clone.neighbors.add(dfs(nei, visited)); } return clone; }
```

5 MULTI-START DFS

★★★★☆

OUTER LOOP DFS

Used In:
- Connected Components
- Number of Provinces
- Number of Islands

```
for (int i = 0; i < n; i++) { if (!visited[i]) { dfs(i, graph, visited); } }
```

BORDER DFS

Used In:
- Pacific Atlantic
- Surrounded Regions

```
// Example for grid top row for (int c = 0; c < n; c++) { dfs(0, c, grid, visited); } // left col for (int r = 0; r < m; r++) { dfs(r, 0, grid, visited); }
```

💡 **The DFS never changes. Only WHERE it starts changes.**

6 WHAT ACTUALLY CHANGES?

★★★★★

Template	Input	Caller	Visited	Return	ONLY THING THAT CHANGES
Graph DFS (Adjacency List)	Adjacency List	`dfs(start)`	`boolean[]`	`void` / `boolean` / `int`	Neighbors come from `graph.get(node)`
Grid DFS	Grid	`dfs(r, c)`	`boolean[][]`	`void` / `int`	Movement using `dirs[]` + boundary checks
Node DFS	Node Graph	`dfs(node)`	`HashMap<Node, Node>`	`Node`	Visited is `HashMap` instead of `boolean[]`
Outer Loop DFS	Adj List / Grid	for all nodes if `!visited` `dfs(i)`	`boolean[]` / `boolean[][]`	`int` / `List` / `void`	Starts DFS from every unvisited node
Border DFS (Multi-Start)	Grid / Graph	for each border run `dfs()`	`boolean[]` / `boolean[][]`	`void`	Starts DFS from borders / multiple sources

💡 **This table is your 30-second interview revision sheet.**

7 RETURN TYPE GUIDE

★★★★★

Return Type	Purpose	Typical Problems	Interview Meaning
void	Marking / Traversal (No direct output)	Flood Fill, Surrounded Regions, Pacific Atlantic	We only visit nodes / cells. No value returned.
boolean	Searching / Decision	Valid Path, Graph Valid Tree, Cycle Detection	Returns true as soon as condition is met.
int	Counting / Aggregation	Number of Islands, Max Area of Island, Count Components	Accumulate and return numeric result.
Node	Graph Construction	Clone Graph	Returns a new structure (node).
List	Collecting Results	All Paths, Topological Sort, Articulation Points	Collect multiple answers and return.

⭐ **Changing the return type usually changes the solution.**

8 AHA MOMENTS

- 🔗 Adjacency List changes only neighbor traversal.
- 🏁 Grid DFS changes only movement using dirs[].
- 🔴 Node Graph changes visited[] into HashMap.
- 🔁 Outer Loop handles disconnected graphs.
- 🟩 Border DFS changes only where DFS starts.
- 📝 **95% of interview problems modify only one or two lines from these templates.**

9 COMMON INTERVIEW MISTAKES

- ❌ Forgot visited[].
- ❌ Forgot boundary check (for grid).
- ❌ Forgot dirs[] for grid movement.
- ❌ Forgot outer loop for disconnected graphs.
- ❌ Forgot HashMap in Clone Graph.
- ❌ Infinite recursion (no base case).
- ❌ Marked visited after recursion.
- ❌ Forgot disconnected graph handling.

10 INTERVIEW THINKING

❓ Problem → 📥 Input Type? → Adjacency List? | Grid? | Node Graph? | Disconnected Graph? | Border / Multi-Start?

Choose Template → **Write Code** 📝

11 PAGE 5 PREVIEW

📖

Next page contains real interview problems.
For each problem, we show:

📄 Base Template → 🔄 Changed Lines → 💡 AHA Moment → ⏱️ Complexity

❓ Problem (Understand) → 📥 Input (What given) → 🧱 Representation (How to model) → 🛠️ Helper Structure (visited, dist, etc.) → ⚙️ Pattern (DFS / BFS / etc.) → 🔧 Tiny Logic Change (Per Problem) → ▶️ Dry Run (Example) → ⏱️ Optimize (Time/Space)

📖 GRAPH PATTERN GUIDE

PAGE 5 — DFS PROBLEM GUIDE (APPLY TEMPLATES)

"Same DFS. Different problems. Only the changing parts."

1. Find the problem.
2. Identify the Base Template (from Page 4).
3. Read the **CHANGED LINES**.
4. Understand the **AHA**.
5. Review the complexity & interview tip.
6. Practice writing from memory.

TEMPLATE LEGEND (From Page 4)

Graph DFS **Gr Grid DFS** **Node DFS** **Outer Loop** **Border DFS**

1 VALID PATH

Base Template: Graph DFS

Problem

Determine if a valid path exists from start to end.

Changed Lines (From Template)

```
boolean dfs(int node, int end, List<List<Integer>> g, boolean[] vis) { if (node == end) return true; // 1. Found destination vis[node] = true; for (int nei : g.get(node)) { if (!vis[nei]) { // 2. Return true immediately if found if (dfs(nei, end, g, vis)) return true; } } return false; // 3. Return false if not found }
```

Caller	Return	Visited
dfs(start)	**boolean**	**boolean[]**

💡 **AHA:** Return true immediately upon finding the target.

Complexity
T: O(V + E)
S: O(V)

Interview Tip
Build adjacency list first.

2 NUMBER OF PROVINCES

Base Template: Graph DFS + Outer Loop

Problem

Count number of connected components.

Changed Lines (From Template)

```
// 1. Outer loop to count components int count = 0; for (int i = 0; i < n; i++) { if (!visited[i]) { count++; dfs(i, graph, visited); } } return count; // dfs() itself is EXACTLY the base Graph DFS template. // (void return type)
```

Caller	Return	Visited
for() → dfs(i)	**void**	**boolean[]**

💡 **AHA:** Each outer loop trigger = 1 new disconnected component.

Complexity
T: O(V + E)
S: O(V)

Interview Tip
Can also use Union Find.

3 NUMBER OF ISLANDS Gr

Base Template: Grid DFS + Outer Loop

Problem

Count number of islands (connected 1s).

Changed Lines (From Template)

```
// 1. Outer loop over grid int islands = 0; for (int r = 0; r < m; r++) { for (int c = 0; c < n; c++) { if (grid[r][c] == '1') { // Unvisited land islands++; dfs(r, c, grid); // Sink the island } } } // 2. Instead of visited[][], mutate grid to '0' // Inside dfs: if(r<0 || c<0 || r>=m || c>=n || grid[r][c]=='0') return; grid[r][c] = '0'; // sink land // explore 4 dirs...
```

Caller	Return	Visited
for() → dfs(r,c)	**void**	**mutate grid**

💡 **AHA:** Mutate input grid ('1' → '0') instead of boolean[][].

Complexity
T: O(M * N)
S: O(M * N)

Interview Tip
Clarify if mutating input is allowed.

4 MAX AREA OF ISLAND Gr

Base Template: Grid DFS + Outer Loop

Problem

Find the maximum area of any island.

Changed Lines (From Template)

```
// 1. Outer Loop Tracks Max Area int maxArea = 0; for (int r = 0; r < m; r++) { for (int c = 0; c < n; c++) { if (grid[r][c] == 1) { maxArea = Math.max(maxArea, dfs(r, c, grid)); } } } // 2. DFS returns area (int) int dfs(int r, int c, int[][] grid) { if (r<0 || c<0 || r>=m || c>=n || grid[r][c]==0) return 0; grid[r][c] = 0; // mark visited int area = 1; // count self for (int[] d : dirs) { area += dfs(r + d[0], c + d[1], grid); } return area; }
```

Caller	Return	Visited
for() → dfs()	**int**	**mutate grid**

💡 **AHA:** Return 1 (self) + sum of 4 neighbors' areas.

Complexity
T: O(M * N)
S: O(M * N)

Interview Tip
DFS returns an int, not void.

5 SURROUNDED REGIONS Gr

Base Template: Grid DFS + Border Loop

Problem

Capture 'O' regions surrounded by 'X'.

Changed Lines (From Template)

```
// 1. Border DFS: Save un-surrounded 'O's by marking them 'S' // loop first & last row if (grid[0][c] == 'O') dfs(0, c, grid); // loop first & last col if (grid[r][0] == 'O') dfs(r, 0, grid); // Inside DFS: grid[r][c] = 'S' // 2. Outer loop to flip the rest for (int r = 0; r < m; r++) { for (int c = 0; c < n; c++) { if (grid[r][c] == 'O') grid[r][c] = 'X'; // capture if (grid[r][c] == 'S') grid[r][c] = 'O'; // restore } }
```

Caller	Return	Visited
Border loop	**void**	**mutate grid**

💡 **AHA:** Reverse thinking. Save safe ones from the borders.

Complexity
T: O(M * N)
S: O(M * N)

Interview Tip
Any 'O' touching a border is safe.

6 CLONE GRAPH

Base Template: Node Graph DFS

Problem

Return a deep copy of the graph.

Changed Lines (From Template)

```
// EXACTLY matches the base Node DFS Template! // 1. Use HashMap for visited & clone mapping HashMap<Node, Node> visited = new HashMap<>(); // 2. Call DFS return dfs(node, visited); // 3. DFS logic Node clone = new Node(node.val); visited.put(node, clone); for (Node nei : node.neighbors) { clone.neighbors.add(dfs(nei, visited)); } return clone;
```

Caller	Return	Visited
dfs(node)	**Node**	**HashMap**

💡 **AHA:** HashMap serves two purposes: visited & clone reference.

Complexity
T: O(V + E)
S: O(V)

Interview Tip
Can also be solved cleanly with BFS.

7 CHANGED LINES SUMMARY (QUICK REFERENCE)

#	Problem	Base Template	Only What Changes
1	**Valid Path**	Graph DFS	Return `true` immediately. Combine results with `||`.
2	**Number of Provinces**	Graph DFS + Outer	Outer loop triggers DFS. Increment count on trigger.
3	**Number of Islands**	Grid DFS + Outer	Outer loop over matrix. Mutate input to sink island.
4	**Max Area of Island**	Grid DFS + Outer	Return type `int`. Add 1 (self) + 4 neighbor calls.
5	**Surrounded Regions**	Grid DFS + Border	Start DFS only from borders. Flip unvisited later.
6	**Clone Graph**	Node DFS	Use `HashMap<Node, Node>` instead of `boolean[] visited`.

- [x] Did I choose the right representation?
- [x] Do I need an outer loop (disconnected)?
- [x] Is my visited[] check correct?
- [x] Did I handle boundaries correctly (Grid)?
- [x] Is my return type correct?
- [x] What is the Time & Space complexity?
- [x] Did I mention mutating input trade-offs?
- [x] Can I optimize further?

❓ Problem (Understand) ➔ 📥 Input (What given) ➔ 🧱 Representation (How to model) ➔ 🛠️ Helper Structure (visited, dist, etc.) ➔ ⚙️ Pattern (DFS / BFS / etc.) ➔ 🔧 Tiny Logic Change (Per Problem) ➔ ▶️ Dry Run (Example) ➔ ⏱️ Optimize (Time/Space)